Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 11 / 22.12.2025

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții
- Funcții de citire/scriere

3. Fișiere text
- Funcții specifice de manipulare

4. Funcții (1)
- Declarare și definire. Apel.
- Metode de trasmitere a paramerilor.
- Pointeri la funcții

5. Tablouri și pointeri
- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite
- Definire și utilizare

9. Funcții (2)
- Funcții cu număr variabil de argumente
- **Preluarea argumentelor funcției main din linia de comandă**
- **Programare generică**



Cursul 11

1. Declarații complexe
2. Preluarea argumentelor funcției main din linia de comandă
3. Programare generică

Chestiuni organizatorice

- *Data propusă* pentru examenul scris:

  30 ianuarie 2025, ora 10.00-12.00

- Regulament de evaluare și notare:

$$Nota = round(min\ (\mathbf{10}, \underbrace{Examen}_{\mathbf{3\text{-}6p}} + \underbrace{Laborator}_{\mathbf{2\text{-}4p}} + \underbrace{Seminar}_{\mathbf{0\text{-}1p}}), 0)$$

*Vezi și: Curs 1, Curs 10

Declarații complexe

- declarație complexă = combinație de pointeri, tablouri și funcții
- combinăm *atributele*:

  () – funcție: int f();

  [] – tablou: int t[20];

  ***** - pointer: int* p;

- importanța parantezelor:
  - ✓ int* f() – f este o funcţie ce returnează un pointer
  - ✓ int *f() – f este o funcţie ce returnează un pointer

    * este un operator prefixat și are o precedență mai mică decât cea a lui ()

  - ✓ int (*f)() - pointer la o funcţie
    - parantezele sunt necesare pentru a impune asocierea corespunzătoare
    - declarațiile nu pot fi citite de la stânga la dreapta

Declarații complexe

Combinații valide de atribute:

- ✓ **int * f()** – funcţie ce returnează un pointer la int
- ✓ **int (*f)()** – pointer la o funcţie fără argumente, ce returnează un int
- ✓ **int *t[10]** – tablou de 10 pointeri la int
- ✓ **int (*p) [13]** – pointer la un tablou de 13 int
- ✓ **int t[5][6]** – tablou bidimensional
- ✓ **int* (*f)()** – pointer la o funcție ce returnează un pointer
- ✓ **int **p** – pointer la un pointer (pointer dublu) la int
- ✓ **int (* p[7]) ( )** – tablou de 7 pointeri la funcții
- ✓ **int (*(*f)())[3][4]** – pointer la o funcţie ce returnează un pointer la un tablou cu 3 linii si 4 coloane de int
- ✓ etc

Declarații complexe

*Combinații **invalide** de atribute*:

[ ] ( ) – funcţie ce returnează un tablou

( ) [ ] – tablou de funcţii

( ) ( ) – funcţie ce returnează o funcţie

- în limbajul C **nu** sunt permise declararea:
  - ✓ unui tablou de funcții
  - ✓ unei funcții care returnează un tablou/o funcție
- dacă vrem ca o funcție să întoarcă rezultatul sub formă de tablou:
  - ✓ transmitem tabloul ca argument prin adresa primului element

    void numeFunctie(int tablou[ ]) sau void numeFunctie(int* tablou)
  - ✓ sau creăm tabloul (în Heap) și întoarcem pointerul corespunzător

    int* numeFunctie()

Declarații complexe

- interpretarea unei declaraţii complexe se face prin înlocuirea *atributelor* (pointeri, funcții, tablouri) prin următoarele *şabloane text*:

Atribut	Șablon text
()	funcția returnează
[n]	tablou de n
*	pointer la

- descifrarea unei declaraţii complexe se face aplicând ***regula dreapta – stânga***, care presupune următorii paşi:
  - se incepe cu identificatorul,
  - se caută în dreapta identificatorului un atribut,
  - dacă nu există, se caută în partea stângă,
  - se substituie atributul cu şablonul text corespunzător,
  - se continuă substituţia dreapta-stânga,
  - se opreşte procesul la întâlnirea tipului datei.

Declarații complexe

- *regula dreapta – stânga*:
  - se incepe cu identificatorul,
  - se caută în dreapta identificatorului un atribut,
  - dacă nu există, se caută în partea stângă,
  - se substituie atributul cu şablonul text corespunzător,
  - se continuă substituţia dreapta-stânga,
  - se opreşte procesul la întâlnirea tipului datei.

int (* a[10]) ( );

- tablou de 10 pointeri la funcţii ce returnează int

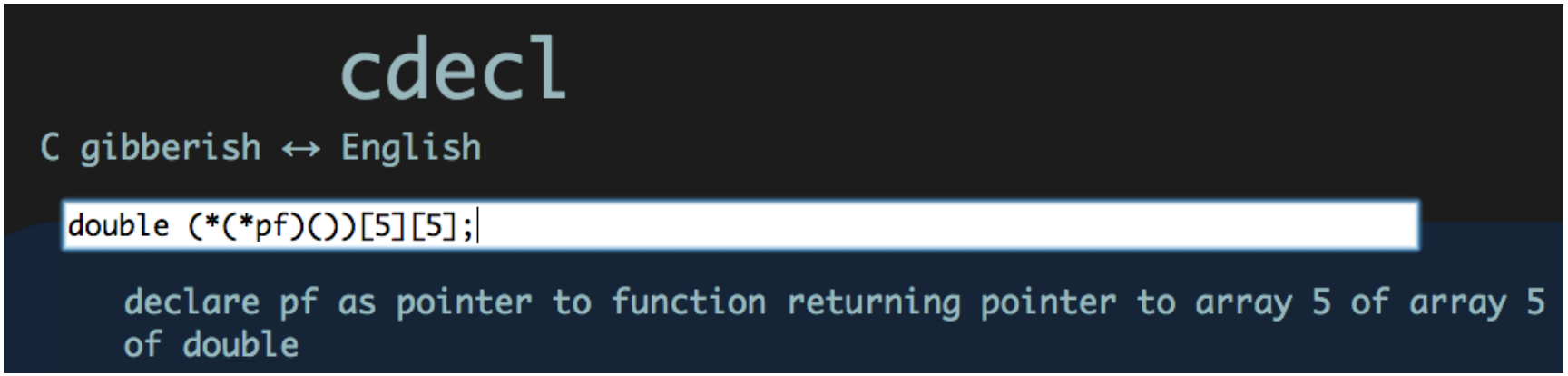
double (*(*pf)())[5][5];

- pointer la o funcţie ce returnează un pointer la un tablou cu 5 linii și 5 coloane de double

Declarații complexe

http://cdecl.org/ - instrument pentru conversia declarațiilor din C în limbaj natural





Cursul 11

1. Declarații complexe
2. **Preluarea argumentelor funcției main din linia de comandă**
3. **Programare generică**

Preluarea argumentelor funcției main din linia de comandă

Un program executabil (comandă) poate fi lansat în execuţie de către interpretorul de comenzi al sistemului de operare.

Exemplu: programul reminder care afişează la terminal un mesaj:

reminder.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("\n Sesiunea incepe pe 22 ianuarie! \n\n");
6      return 0;
7  }
```

- program fără parametri, lansat în execuţie prin: **>./reminder**

Preluarea argumentelor funcției main din linia de comandă

- un program poate avea şi parametri, care apar după numele comenzii şi sunt separaţi prin spaţii libere.
- de exemplu, programul **reminder** ar putea avea un parametru şir de caractere, fiind lansat în execuţie, în acest caz prin:

  > ./reminder parametru
- programul poate avea acces la parametrii liniei de comandă, dacă funcţia **main()** prezintă argumente:

  int main(int argc, char *argv[]) { … }
- primul argument, **argc** (*argumentul contor*) este o variabilă de tip întreg care reprezintă *numărul de parametri ai programului/ comenzii*
- al doilea argument, **argv** (*argumentul vector*) este un pointer la un tablou de pointeri la şiruri de caractere, care conţin argumentele (fiecare element al tabloului = un pointer la un şir de caractere = un argument).

Preluarea argumentelor funcției main din linia de comandă

- primul argument, **argc** (*argumentul contor*)
- al doilea argument, **argv** (*argumentul vector*)
- **argv[0]** conţine întotdeauna *numele programului;*
- **argv[1]** conţine *primul parametru* ca șir de caractere;
- **argv[2]** conţine *al doilea parametru* ca șir de caractere;
- **argv[argc-1]** conţine *ultimul parametru* ca șir de caractere;
- **argv[argc]** va fi un pointer la un şir vid (**NULL**);

- pentru o comandă fără parametri: **argc=1**,
- iar o comandă cu 2 parametri va avea **argc=3**.

Cursul 11

1. Declarații complexe
2. Preluarea argumentelor funcției main din linia de comandă
3. **Programare generică**

Pointeri generici

- **pointer generic = pointer la tipul void**
- poate stoca adresa de memorie a unui obiect oarecare
- dimensiunea zonei de memorie indicate și interpretarea informației conținute nu sunt definite
- nu poate fi dereferențiat

```c
#include <stdio.h>
int main(){
    int i = 6;
    void *p;
    p = &i;
    printf("p pointeaza catre o valoare intreaga %d\n", *p);
    return 0;
}
```

input stderr

Compilation failed due to following error(s).

```
main.c: In function ‘main’:
main.c:6:57: warning: dereferencing ‘void *’ pointer
    6 |     printf("p pointeaza catre o valoare intreaga %d\n", *p);
      |                                                         ^~
main.c:6:57: error: invalid use of void expression
```

Pointeri generici

- pointer la tipul void (pointer generic)
- poate stoca adresa de memorie a unui obiect oarecare
- dimensiunea zonei de memorie indicate și interpretarea informației conținute nu sunt definite
- nu poate fi dereferențiat
- pentru a-l folosi (la dereferențiere) trebuie convertit la un alt tip de pointer prin operatorul cast (tip *)
- un singur pointer poate pointa la diferite tipuri de date la momente diferite pe parcursul execuției unui program

Pointeri generici

- pentru a folosi (la dereferențiere) un pointer generic trebuie convertit la un alt tip de pointer prin **operatorul cast (tip *)**
- un singur pointer poate pointa la diferite tipuri de date la momente diferite pe parcursul execuției unui program

```c
#include <stdio.h>

int main()
{
    int i = 6;
    char c = 'a';
    void *p;

    p = &i;
    printf("p pointeaza catre valoarea intreaga %d\n", *(int*)p);

    p = &c;
    printf("p pointeaza catre caracterul %c\n", *(char*)p);

    return 0;
}
```

```
p pointeaza catre valoarea intreaga 6
p pointeaza catre caracterul a
```

Pointeri generici

Pointer la tipul void (pointer generic):

```c
#include <stdio.h>

int main()
{
    int i = 300;
    void *p;

    p = &i;
    printf("Valoare pointer void = %p\n", p);
    printf("Valoare pointer dereferentiat ca int = %d\n", *(int*)p);
    printf("Valoare pointer dereferentiat ca char = %d\n", *(char*)p);

    float v[5] = {1.2, 0.4, 0.6, 6.2, 5.1};
    p = v;
    printf("Valoare pointer = %.2f\n", ((float*)p)[3]);

    return 0;
}
```

```
Valoare pointer void = 0x7ffe00308b74
Valoare pointer dereferentiat ca int = 300
Valoare pointer dereferentiat ca char = 44
Valoare pointer = 6.20
```

330 in binar = 0000 0001 0010 1100
44 in binar = 0010 1100

Programare generică

- **programare generică = paradigmă de programare** în care codăm diferiți algoritmi pentru tipuri de date generice (neinstanțiate)

- exemplu: un algoritm de sortare al unui tablou de numere arată la fel și pentru date de tipul char, int, float, double => putem particulariza funcția de comparare să lucreze cu date de tipul char, int, float, double.

- în programare generică folosim pointeri generici

Aplicație: Să presupunem că vrem să construim funcția de căutare secvențială a unui element într- un vector de întregi; functia va returna pozitia elementului sau –1 dacă el nu există.

Funcția primește ca parametri:

- vectorul (int *v),
- numărul de elemente (int n) și
- elementul (valoarea) de căutat (int k).

```
int cauta(int *v, int n, int k)
{
    for(int i = 0; i < n; i++)
        if( v[i] == k )
            return i;
    return -1;
}
```

Programare generică

Exemplul 1: Scrieți **o funcție generică** de căutare care să returneze un pointer generic către prima apariție a valorii **c** în tabloul unidimensional **v** format din **n** elemente, fiecare având dimensiunea **dim** octeți, sau pointerul **NULL** dacă valoarea **c** nu se găsește în tablou.

```
void* cauta(const void* v, const void* c, int n, int dim)
{
    char *p = (char*)v;
    for(int i = 0; i < n; i++)
        if(!memcmp(c, p+i*dim, dim)) return p+i*dim;
    return NULL;
}
```

Funcția **memcmp**: *compare bytes in memory*

```
#include <string.h>

int memcmp(const void *s1, const void *s2, size_t n);
```

- The *memcmp*() function shall compare the first *n* bytes (each interpreted as **unsigned char**) of the object pointed to by *s1* to the first *n* bytes of the object pointed to by *s2*.
- The sign of a non-zero return value shall be determined by the sign of the difference between the values of the first pair of bytes (both interpreted as type **unsigned char**) that differ in the objects being compared.

Pentru a nu fi dependentă de tipul de dată al vectorului, funcția va fi reconstruită astfel încât vectorul să fie primit prin intermediul unui pointer la void.

Lucrul cu o adresă spre un tip nedefinit are implicații în sensul că e necesar să se transmită un parametru suplimentar care să exprime lungimea tipului de dată al elementelor vectorului.

```
int cauta(void *v, int n, int dim_el, void *el)
{
    char *ec = (char*)v;
    for(int i = 0; i < n; i++)
        if(!memcmp(ec + i*dim_el, el, dim_el))
            return i;
    return -1;
}
```

unde **int dim_el =** dimensiunea în baiti a tipului elementelor vectorului, elementul de cautat este dat prin adresa lui (parametrul void *el), lungimea se considera a fi aceeasi cu a elementelor vectorului.

Se observa ca pointerul primit în functie (v) este convertit din void* în char* pentru că dimensiunea unui caracter este de un octet, iar dimensiunea unui element (dim_el) este exprimată în baiti.

Compararea dintre elemente se face folosindu-se functia de biblioteca memcmp (al cărei prototip se află în fisierul header **memory.h**) care primeste doua adrese si lungimea în baiti a zonei de memorie de comparat.

Aplicație:

```c
#include <stdio.h>
#include <string.h>
#include <memory.h>

int cauta(void* v, int n, int dim_el, void *el) {
    char *p = (char*)v;
    for(int i = 0; i < n; i++)
        if(!memcmp(p+i*dim_el, el, dim_el)) return i;
    return -1;
}

int main() {
    int k;
    int a[] = {3, 6, 4, 1, 8}; int t = 4;
    printf("Elementul %d este ", t);
    (k = cauta(a, sizeof(a)/sizeof(int), sizeof(int), &t)) == -1 ?
        printf("inexistent!\n") : printf("pe pozitia %d.\n", k);

    double b[] = {3.5, 6.2, 4.9, 1, 28.15}, r = 6.2;
    printf("Elementul %.2f este ", r);
    (k = cauta(b, sizeof(b)/sizeof(double), sizeof(double), &r)) == -1 ?
        printf("inexistent!\n") : printf("pe pozitia %d.\n", k);

    char *sir = "Un sir de caractere", c = 'f';
    printf("Elementul %c este ", c);
    (k = cauta(sir, strlen(sir), sizeof(char), &c)) == -1 ?
        printf("inexistent!\n") : printf("pe pozitia %d.\n", k);

    return 0;
}
```

Elementul 4 este pe pozitia 2.
Elementul 6.20 este pe pozitia 1.
Elementul f este inexistent!

Programare generică

Exemplul 1. o funcție generică de căutare

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <memory.h>
4
5  void* cauta(const void* v, const void* c, int n, int dim)
6  {
7      char *p = (char*)v;
8      for(int i = 0; i < n; i++)
9          if(!memcmp(c, p+i*dim, dim)) return p+i*dim;
10     return NULL;
11 }
12
13 int main()
14 {
15     int v[5] = {3, 0, 6, 2, 5};
16     int s = 2, *p, n = sizeof(v)/sizeof(int);
17     p = (int*)cauta(v, &s, n, sizeof(int));
18     if(p)
19         printf("Primul %d se afla pe pozitia %d.\n", s, p-v);
20
21     return 0;
22 }
```

```
Primul 2 se afla pe pozitia 3.
```

Programare generică

Exemplul 1. o funcție generică de căutare

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <memory.h>
4
5  void* cauta(const void* v, const void* c, int n, int dim)
6  {
7      char *p = (char*)v;
8      for(int i = 0; i < n; i++)
9          if(!memcmp(c, p+i*dim, dim)) return p+i*dim;
10     return NULL;
11 }
12
13 int main()
14 {
15     char v[] = "Un sir de caractere oarecare";
16     char ch = 'e', *p;
17     p = (char*)cauta(v, &ch, sizeof(v), 1);
18     if(p)
19         printf("Primul %c se afla pe pozitia %d.\n", ch, p-v);
20
21     return 0;
22 }
```

```
Primul e se afla pe pozitia 8.
```

Din Cursul 6: Utilitatea pointerilor la funcții

- se folosesc în programarea generică, realizăm apeluri de tip callback
- o funcţie C transmisă, printr-un pointer, ca argument unei alte funcţii F se numeşte şi funcţie “callback”, pentru că ea va fi apelată “înapoi” de funcţia F

Exemple:

int suma(int n, int (*expresie)(int)); //sumă generică de n numere

void qsort(void *adresa,int nr_elemente, int dimensiune_element, int (*cmp)(const void *, const void *)) ; //funcția qsort din stdlib.h

Cursul 6: Utilitatea pointerilor la funcții

Exemplul 2: Să se calculeze suma $S_k(n) = \sum_{i=1}^{n} i^k$

$$S_1(n) = 1 + 2 + \ldots + n$$

$$S_2(n) = 1^2 + 2^2 + \ldots + n^2$$

$$S_k(n) = \sum_{i=1}^{n} expresie(i)$$

Folosind pointeri la funcții pot să văd funcția ca o variabilă

Cursul 6: Utilitatea pointerilor la funcții

Exemplul 2: Să se calculeze suma $S_k(n) = \sum_{i=1}^{n} i^k$ pentru k={1,2}.

```c
#include <stdio.h>
#include <stdlib.h>

int suma(int n, int (*expresie)(int))
{
    int s = 0;
    for(int i = 1; i <= n; i++)
        s = s + expresie(i);
    return s;
}

int expresie1(int x) { return x;}

int expresie2(int x) { return x * x;}

int main()
{
    int S1 = suma(5, expresie1);
    printf("S1 = %d\n", S1);
    int S2 = suma(5, expresie2);
    printf("S2 = %d\n", S2);

    return 0;
}
```

S1 = 15
S2 = 55

15 = 1 + 2 + 3 + 4 + 5
55 = 1 + 4 + 9 + 16 + 25

Cursul 6: Utilitatea pointerilor la funcții

- funcția **qsort** din **stdlib.h** folosită pentru *sortarea unui vector/tablou*
- antetul lui **qsort** este:

void qsort (void *adresa, int nr_elemente, int dimensiune_element,

int (*cmp) (const void *, const void *))

unde:

- ✓ **adresa** = pointer la adresa primului element al tabloului ce urmeaza a fi sortat (pointer generic – nu are o aritmetică inclusă)
- ✓ **nr_elemente** = numarul de elemente al vectorului
- ✓ **dimensiune_element** = dimensiunea in octeți a fiecărui element al tabloului (char = 1 octet, int = 4 octeți, etc)
- ✓ **cmp** = funcția de comparare a două elemente

Programare generică - qsort

- funcția **qsort** din **stdlib.h** - pentru sortarea unui vector/tablou

void qsort (void *adresa, int nr_elemente, int dimensiune_element,

int (*cmp) (const void *, const void *))

int cmp(const void *a, const void *b)



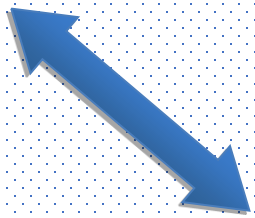
adresele a două elemente din tablou

- **cmp** este o funcție generică comparator, compară 2 elemente de orice tip. Întoarce:
  - un număr < 0 dacă vrem a la stânga lui b
  - un număr >0 dacă vrem a la dreapta lui b
  - 0, dacă nu contează

Programare generică - qsort

Exemplu de funcție cmp pentru sortarea unui vector de numere întregi:

```
int cmp(const void *a, const void *b)
{
    int va, vb;
    va = *(int*)a;
    vb = *(int*)b;
    if(va < vb) return -1;
    if(va > vb) return 1;
    return 0;
}
```



```
int cmp(const void *a, const void *b)
{
    return *(int*)a - *(int*)b;
}
```

Programare generică - qsort

Exemplu: sortarea unui vector de numere întregi:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int cmp(const void* a, const void * b)
5  {
6      return *(int *)a - *(int *)b;
7  }
8
9  int main()
10 {
11     int v[] = {10, 0, 1, -6, 12, 3, 1, 5};
12     int n = sizeof(v)/sizeof(int);
13     qsort(v, n, sizeof(int), cmp);
14     for(int i = 0; i < n; i++)
15         printf("%d ", v[i]);
16     printf("\n");
17
18     return 0;
19 }
```

```
-6 0 1 1 3 5 10 12
```

Programare generică

Exemplul 3: Scrieți o funcție generică de căutare care să returneze un pointer generic către prima apariție a valorii **x** în tabloul unidimensional **t** format din **n** elemente, fiecare având dimensiunea **d** octeți, sau pointerul NULL dacă valoarea **x** nu se găsește în tablou.
Folosiți o funcție comparator **cmp** cu antetul

int (cmpValori*)(const void*, const void *)

care returnează valoarea 1 dacă valorile aflate la adresele primite ca parametri sunt egale sau 0 în caz contrar.
Explicitarea funcţiei concrete de comparaţie se va face în apel.

Vom considera două cazuri:

-compararea a două valori întregi,

-compararea a două şiruri de caractere.

Programare generică

Exemplul 3: o funcție generică de căutare care să returneze un pointer generic către prima apariție a valorii **x** în tabloul unidimensional **t** format din **n** elemente, fiecare având dimensiunea **d** octeți, sau pointerul NULL dacă valoarea **x** nu se găsește în tablou.

```
1  #include <stdio.h>
2  #include <memory.h>
3
4  void* cauta(const void *x, const void *t, int n, int d, int (*f)(const void*, const void*))
5  {
6      char *p = (char*)t;
7      for(int i = 0; i < n; i++)
8          if(f(x, p + i * d))
9              return p + i * d;
10     return NULL;
11 }
12
13 int cmpInt (const void* a, const void * b){
14     return *((int *)a) == *((int *)b);
15 }
16
17 int cmpChar (const void* a, const void * b){
18     return *((char *)a) == *((char *)b);
19 }
```

Programare generică

Exemplul 3:

```
21  int main()
22  {
23      int v[] = {10, 1, 12, 3, 1, 5};
24      int x; printf("valoare cautata = "); scanf("%d", &x);
25      int *p;
26      p = (int*)cauta(&x, v, sizeof(v)/sizeof(int), sizeof(int), cmpInt);
27      if(p) printf("%d se afla in tabloul v pe pozitia %d. \n", x, p - v);
28       else printf("%d NU se afla in tabloul v. \n", x);
29
30      char w[] = "Orice student intelege programarea generica.";  getchar();
31      printf("\ncaracter cautat = "); char ch = getchar();
32      char *q;
33      q  = (char*)cauta(&ch, w, sizeof(w)/sizeof(char), sizeof(char), cmpChar);
34      if(q) printf("%c se afla in tabloul w pe pozitia %d. \n", ch, q - w);
35       else printf("%c NU se afla in tabloul w. \n", ch);
36
37      return 0;
38  }
```

valoare cautata = 5
5 se afla in tabloul v pe pozitia 5.

caracter cautat = d
d se afla in tabloul w pe pozitia 9.

Cursul 11

1. **Declarații complexe**
2. **Preluarea argumentelor funcției main din linia de comandă**
3. **Programare generică**

Cursul 12

1. **Recapitulare**
2. *Exemple de subiecte*

Exemple

1. Reprezentati pe 32 de biti numerele intregi 1234 si – 5678.
2. Analizati programul urmator. Daca este corect spuneti ce se va afisa, in caz contrar explicati de unde provine eroarea.

```
#include <stdio.h>
int v[10];
int main() {
int *pv, *pv1, i;
pv = v;
*(pv + 1) = 10; *(pv + 4) =4; *(pv - 3) = 1;
pv1 = &(*(pv++)); *(pv1+5) =11; *(pv1+8) = v[2] - 4;
for(i=0; i<9; i++) printf("%d ", i+v[i]);}
```

3. Ce valoare va avea variabila x in urma executarii secventei de cod de mai jos? Justificați.

```
int n = 100, x;
x = n | (40 & 10) && n>>(4<<2) + n | (10 ^ 8);
```

Exemple

4. Fișierul text $note.txt$ conține pe prima linie un număr natural $n \geq 1$, iar pe fiecare din următoarele n linii informații despre nota obținută de un student la un examen, astfel:

nume student, prenume student, grupa, disciplina, nota obținută

Definiți o structură $Student$ care să permită stocarea informațiilor referitoare la un student și, eventual, a altor informații pe care le considerați necesare pentru rezolvarea cerințelor de mai jos, în mod optim din punct de vedere al memoriei utilizate.

a) Afișați numărul studenților care au promovat examenul la o disciplină d, citită de la tastatură, precum și numărul studenților care au susținut examen la disciplina respectivă.

b) Folosind un tablou unidimensional cu elemente de tip $Student$ și funcția qsort din biblioteca stdlib.h, calculați media fiecărui student și afișați studenții în ordinea descrescătoare a mediilor, iar în cazul unor medii egale în ordine alfabetică.

c) Scrieți într-un fișier binar informațiile despre studenții care au mediile maxime. Numele fișierului binar se va citi de la tastatură.

Cursul 12: 12.01.2026

Examen: 30.01.2025